

# Machine Learning Analysis Report: Predicting Defective Software Modules

Udacity Master's in AI, Applied Machine Learning Capstone

Belal Nwiran

## Overview

This project predicts whether a software module is defective, using measurements taken from its source code. I used the Software Defect Prediction dataset from Kaggle (file jm1.csv; <https://www.kaggle.com/datasets/semustafacevik/software-defect-prediction>). The task is supervised binary classification. I compared a baseline, a logistic regression model, and a random forest model, and judged them mainly with precision, recall, and F1-score.

The project's GitHub repo: <https://github.com/bnwiran/applied-machine-learning-project>

## Dataset Description

Each row in the dataset is one code module, described by 21 static code metrics such as lines of code, complexity measures, and Halstead measures. The target column, defects, is True when the module was defective. The original file has 10,885 rows and 22 columns. This is the JM1 data set, one of the NASA defect data sets that are widely used in defect prediction research (Shepperd et al., 2013). After cleaning, 8,907 rows remained, and 22.49% of them were defective. I split these rows 80/20 with stratification, which gave 7,125 training rows and 1,782 test rows.

## Modeling Approach

### Data Preparation

Five columns were read as text because they contained “?” marks, so I converted them to numbers. The five rows that stayed missing were dropped. I also removed 1,973 exact duplicate rows, because copies of the same row in both the training and test sets would make test scores look better than they are. The NASA data sets are known to have quality problems, and researchers are advised to describe their preprocessing clearly (Shepperd et al., 2013), so each step is documented in the notebook.

In 982 rows (11.03%), every token-based Halstead metric is 0, while size and structure metrics such as lines of code are filled in. This looks like a measurement gap, but that is my own inference and the dataset does not document it. These rows also have a higher defect rate (30.96%) than the whole data (22.49%). I kept them, set their Halstead values to missing, filled the gaps with the training median, and kept a `halstead_missing` flag column so the model can still use this signal. Both models received the same treatment. For logistic regression, I also applied a log transform and standardization, because the metrics have very different scales and this model is sensitive to that. All steps ran inside a scikit-learn pipeline, so they were fitted on the training data only and no information from the test set was used.

## Model Choice

I chose two models with different strengths. Logistic regression is a simple linear model, which gives an easy-to-explain comparison point. A random forest combines the votes of many decision trees (Breiman, 2001), which lets it fit non-linear patterns without feature scaling. Breiman (2001) also showed that adding more trees does not make a forest overfit more. Both models used balanced class weights, because only about 22% of modules are defective. I tuned a few settings (the regularization strength for logistic regression, and the tree depth and minimum leaf size for the random forest, which used 300 trees) with 5-fold stratified cross-validation on the training set, using F1 as the score. The test set was used only once, at the end.

## Evaluation Metrics

Only 22.5% of the test modules are defective, so a model that always predicts “clean” is about 77.5% accurate and still finds nothing. Accuracy is therefore not a good main metric here. I used precision (the share of flagged modules that are truly defective), recall (the share of defective modules that were found), and F1-score (the harmonic mean of the two) (scikit-learn developers, n.d.). On imbalanced data, precision-recall evaluation is more informative than ROC-based evaluation (Saito & Rehmsmeier, 2015), so I also used precision-recall curves and average precision.

## Assumptions

Logistic regression is a linear model, so it assumes that a weighted sum of the log-scaled features separates defective from clean modules reasonably well. The random forest does not need this assumption. Both models assume that the rows are independent, that the test data looks like the training data, and that the labels are mostly correct. I used the default 0.5 decision threshold and did not tune it.

## Results

Table 1 shows the test-set results, and Table 2 compares training, cross-validation, and test F1. The confusion matrices and precision-recall curves are in the Evaluation section of `modeling.ipynb`.

**Table 1**

*Test-Set Performance*

| Model                     | Accuracy | Precision | Recall | F1    | Average precision |
|---------------------------|----------|-----------|--------|-------|-------------------|
| Baseline (always “clean”) | 0.775    | 0.000     | 0.000  | 0.000 | 0.225             |
| Logistic regression       | 0.670    | 0.361     | 0.603  | 0.451 | 0.407             |
| Random forest             | 0.705    | 0.393     | 0.574  | 0.467 | 0.417             |

**Note.** Precision, recall, and F1 are for the defective class. The test set has 1,782 modules, and 401 of them are defective.

The baseline scored 0 on precision, recall, and F1, although its accuracy was 0.775. Logistic regression reached an F1 of 0.451 (precision 0.361, recall 0.603), and the random forest reached an F1 of 0.467 (precision 0.393, recall 0.574). Both models have lower accuracy than the baseline (0.670 and 0.705), because they flag many modules as defective. Both are clearly above the baseline in average precision (0.407 and 0.417, against 0.225, which equals the defect rate).

**Table 2**

*F1-Score on Training, Cross-Validation, and Test Data*

| Model               | Training F1 | Cross-validation F1 | Test F1 |
|---------------------|-------------|---------------------|---------|
| Logistic regression | 0.447       | 0.447 ± 0.014       | 0.451   |
| Random forest       | 0.628       | 0.457 ± 0.019       | 0.467   |

*Note.* Cross-validation used 5 stratified folds on the training set (mean ± standard deviation).

Test F1 is close to cross-validated F1 for both models, so the results are stable. The random forest's training F1 (0.628) is higher than its cross-validation and test F1, which shows some overfitting, while logistic regression scores about the same on all three. The difference between the two models is small. In cross-validation it was 0.010 F1, which is smaller than the variation between folds. The main difference is the trade-off: logistic regression finds slightly more defective modules (recall 0.603 against 0.574), while the random forest is more precise (0.393 against 0.361) and leaves more clean modules unflagged (74% against 69%).

## Interpretation for a Non-Technical Audience

The model reads the code measurements of a module and flags it as likely to contain a defect. I tested it on 1,782 modules with known outcomes. Of every 100 modules the random forest flagged, about 39 really were defective, and for logistic regression it was about 36. Of every 100 defective modules, the random forest found about 57 and logistic regression found about 60. A random pick would be right only about 22 times in 100, so flagged modules are about 1.6 to 1.7 times more likely to be defective than an average module.

Even so, most flags are false alarms, and roughly 4 in 10 defective modules are missed. The model is therefore useful for deciding which modules to review first, and it should not replace testing or code review. If review time is limited, the random forest is the better fit because it raises fewer false alarms. If missing a defect is more costly, logistic regression is the better fit because it finds slightly more. A model that always says “clean” looks 77.5% accurate but finds no defects, which is why accuracy alone was not used.

## Limitations and Potential Bias

Performance is modest. With precision below 0.40, most flagged modules are clean, and with recall of 0.57 to 0.60, about 4 in 10 defective modules are missed. The model uses only static code metrics, so it

cannot see other things that may drive defects, such as who wrote the code, how often it changed, or how well it was tested. Data quality is another limit. The 982 rows with all-zero Halstead values may be a measurement artifact, so the model's use of the flag may reflect how the data was collected and not real code risk. Finally, I used a single train/test split and did not tune the 0.5 threshold, so the scores are estimates with some uncertainty (the cross-validation standard deviation was 0.014 to 0.019).

The main source of bias is representativeness. All rows come from one collection of NASA projects, so the model may learn patterns that are specific to those projects and may not carry over to other teams, languages, or code bases. There is also a responsible-use concern: a defect score could be misused to judge developers, even though the data has no developer information. To reduce these risks, I documented the data source and every preprocessing step, as recommended by Shepperd et al. (2013). Before real use, I would also test the model on data from a different project, and use the scores only to prioritize reviews, with people making the final decisions.

## Note on AI Assistance

I used Claude (Anthropic) to plan the workflow and to help draft the model training and evaluation code, the notebook summary, and the first version of this report. I ran the code myself, checked the numbers against my notebook outputs, revised the text, and I am responsible for the final content and decisions.

## References

Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.

<https://doi.org/10.1023/A:1010933404324>

Saito, T., & Rehmsmeier, M. (2015). The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLOS ONE*, 10(3), Article e0118432.

<https://doi.org/10.1371/journal.pone.0118432>

scikit-learn developers. (n.d.). Metrics and scoring: Quantifying the quality of predictions. scikit-learn.

Retrieved September 29, 2026, from [https://scikit-learn.org/stable/modules/model\\_evaluation.html](https://scikit-learn.org/stable/modules/model_evaluation.html)

Shepperd, M., Song, Q., Sun, Z., & Mair, C. (2013). Data quality: Some comments on the NASA software defect datasets. *IEEE Transactions on Software Engineering*, 39(9), 1208–1215.

<https://doi.org/10.1109/TSE.2013.11>